GC-F-004v.09

COLOMBIA POTENCIA DE LA VIDA

SENA

Análisis y desarrollo de software.

Instructor

Ing. Diego Fernando Calderon Silva
Correo: dfcalderon@sena.edu.co

Introducción NestJs + Ts



¿Qué es NestJs?

NestJS es un framework progresivo de Node.js que facilita la construcción de aplicaciones del lado del servidor eficientes, escalables y mantenibles. Desarrollado en TypeScript, combina elementos de la programación orientada a objetos, funcional y reactiva, proporcionando una estructura modular que mejora la organización del código.



NestJS es ampliamente adoptado por empresas de renombre, incluyendo Sanofi, Adidas, Autodesk, Mercedes-Benz, GitLab, Red Hat, BMW, Roche, IBM, Decathlon, Société Générale, JetBrains, TotalEnergies, Capgemini y REWE digital, entre otras.

Para facilitar el desarrollo, NestJS ofrece una CLI (Interfaz de Línea de Comandos) que ayuda a inicializar, desarrollar y mantener aplicaciones Nest, promoviendo buenas prácticas arquitectónicas y una estructura bien organizada.

Instalar Nest CLI

Si vamos a la documentación oficial nos indica que para instalar debemos usar el comando .

```
$ npm install -g @nestjs/cli
```

Recuerda que siempre que usemos **–g** significa que haremos una instalación de forma global.

```
ingdiegoc@iMac-de-Diego ~ % npm install -g @nestjs/cli

added 263 packages in 17s

48 packages are looking for funding
  run `npm fund` for details
```

```
ingdiegoc@iMac-de-Diego Nest % nest -v
11.0.5
```

Generar el primer proyecto

Ahora vamos a crear un proyecto, para empezar escribimos, y ejecutamos localhost:3000 por defecto.

Nuevo proyecto: en el path actual

```
nest new project-name
```

```
ingdiegoc@iMac-de-Diego Nest % nest new car-dealership
[✨  We will scaffold your app in a few seconds..

✔ Which package manager would you ❤️to use? npm
CREATE car-dealership/.prettierrc (51 bytes)
CREATE car-dealership/README.md (5020 bytes)
CREATE car-dealership/eslint.config.mjs (856 bytes)
CREATE car-dealership/nest-cli.json (171 bytes)
CREATE car-dealership/package.json (2040 bytes)
CREATE car-dealership/tsconfig.build.json (97 bytes)
CREATE car-dealership/tsconfig.json (544 bytes)
CREATE car-dealership/src/app.controller.ts (274 bytes)
CREATE car-dealership/src/app.module.ts (249 bytes)
CREATE car-dealership/src/app.service.ts (142 bytes)
CREATE car-dealership/src/main.ts (228 bytes)
CREATE car-dealership/src/app.controller.spec.ts (617 bytes)
CREATE car-dealership/test/jest-e2e.json (183 bytes)
CREATE car-dealership/test/app.e2e-spec.ts (674 bytes)

✔ Installation in progress... ☕

🚀  Successfully created project car-dealership
👉  Get started with the following commands:

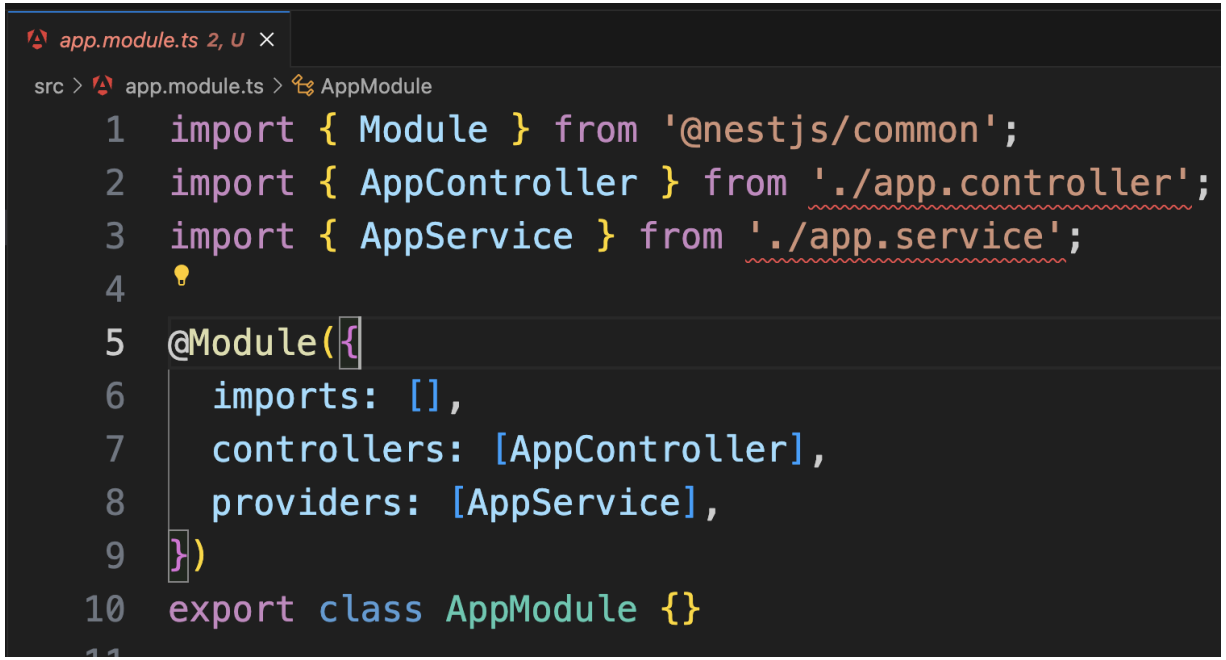
$ cd car-dealership
$ npm run start


                        Thanks for installing Nest 🙏
                Please consider donating to our open collective
                       to help us maintain this package.


🍷 Donate: https://opencollective.com/nest
```

Módulos

Vamos a fijarnos en la carpeta src del proyecto donde hay varios archivos ya definidos, pero por ahora solo nos importa dejar main y app.module.ts asi que elimino lo demás.



app.modules.ts nos mostrara el error clásico de que no encuentra la importación y es natural ya que acabamos de borrar esos archivos.

Módulos

Voy a dejar el archivo de app.modules.ts vacio, y voy a crear un nuevo modulo de exports así:

app.module.ts U

src > app.module.ts > ...

```ts
import { Module } from '@nestjs/common';

@Module({
  imports: [],
  controllers: [],
  providers: [],
  exports: [],
})
export class AppModule {}
```

Ahora la pregunta es… ¿Qué son los modulos?

Módulos

Los módulos agrupan y desacoplan un conjunto de funcionalidad especifica por dominio. Ej: auth.module.ts, encargado de todo lo relacionado a la autenticación.

En **NestJS**, los **módulos** son una parte fundamental del framework, ya que ayudan a organizar y encapsular el código de una manera modular y reutilizable.

Un **módulo** es una clase decorada con @Module(), que agrupa **controladores, servicios, proveedores y otros módulos** relacionados en una sola unidad lógica.

app.module.ts 2, U

src > app.module.ts > ...

```ts
import { Module } from '@nestjs/common';
import { UsersController } from './users.controller';
import { UsersService } from './users.service';

@Module({
  controllers: [UsersController],  // Define los controladores del módulo
  providers: [UsersService],       // Define los servicios o proveedores
  exports: [UsersService],         // Permite que otros módulos usen este servicio
})
export class UsersModule {}        // Declaración del módulo
```

Módulos

Para el ejemplo anterior:

@Module() → Es el decorador que indica que esta clase es un módulo.

controllers: [UsersController] → Lista de los controladores del módulo.

providers: [UsersService] → Lista de servicios o proveedores disponibles en este módulo.

exports: [UsersService] → Hace que el servicio UsersService pueda ser usado en otros módulos.

UsersModule → Es el nombre del módulo.

app.module.ts 2, U

src > app.module.ts > ...

```ts
import { Module } from '@nestjs/common';
import { UsersController } from './users.controller';
import { UsersService } from './users.service';

@Module({
  controllers: [UsersController],  // Define los controladores del módulo
  providers: [UsersService],       // Define los servicios o proveedores
  exports: [UsersService],         // Permite que otros módulos usen este servicio
})
export class UsersModule {}        // Declaración del módulo
```

Módulos

Ahora, volviendo a nuestro código base. Debemos saber que el module principal debe tener acceso a todos los demás modulos.
Veamos ahora el main que es el punto de entrada de nuestra app.

app.module.ts U | main.ts U

src > main.ts > ...

```ts
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  await app.listen(process.env.PORT ?? 3000);
}
bootstrap();
```

Si nos fijamos, hay una función que se llama bootstrap, esto no hace referencia al framework de css que da estilos, es una función que se puede cambiar por cualquier nombre, pero generalmente la gente nunca lo hace, esa función es la que se ejecuta al inicial el proyecto.

Módulos

Si nos fijamos un poco mas, vemos en la línea 5 que dice que nuestra función de boostrap que se inicia al ejecutar el proyecto, va a pasarle nuestro AppModule, es decir el modulo que tenemos creado el cual le va a decir a nest que dependencias, servicios, controladores, etc.

app.module.ts U | main.ts U

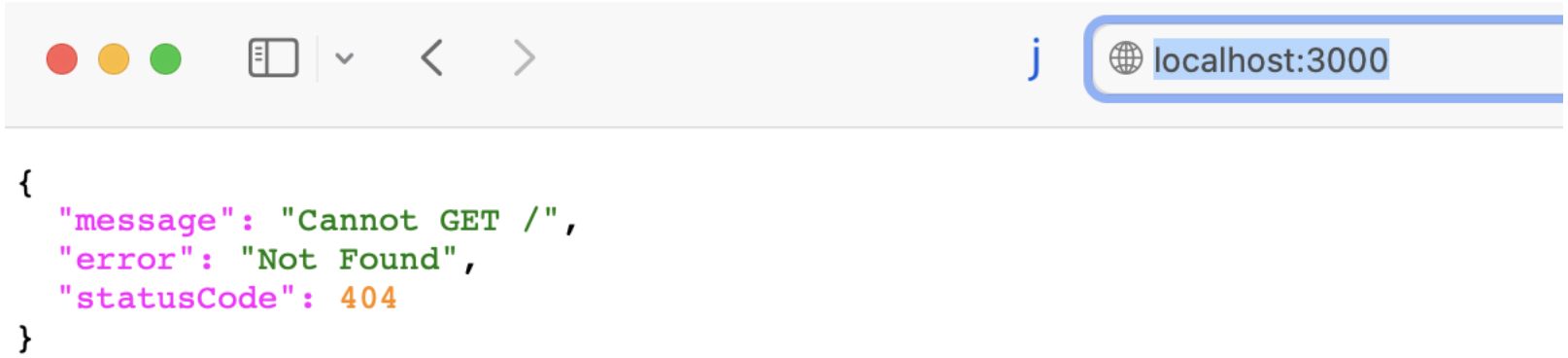
src > main.ts > ...

```ts
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  await app.listen(process.env.PORT ?? 3000);
}
bootstrap();
```

Módulos

Ahora si voy a mi navegador (JASON viewer pro), en la ruta http://localhost:3000 me encontrare esto.



Claramente el navegador no es la mejor opción para visualizar este tipo de respuestas por lo que de ahora en adelante lo hare en postman

Controladores

Los controladores son aquellos que controlan rutas, son los encargados de escuchar la solicitud y emitir una respuesta. Ej: Rutas CRUD

En **NestJS**, los **controladores** son responsables de manejar las **solicitudes HTTP** y devolver respuestas al cliente. Se utilizan para definir las rutas y la lógica que se ejecuta cuando un cliente hace una petición a la API.

NestJS permite manejar diferentes tipos de peticiones HTTP con los siguientes **decoradores**:

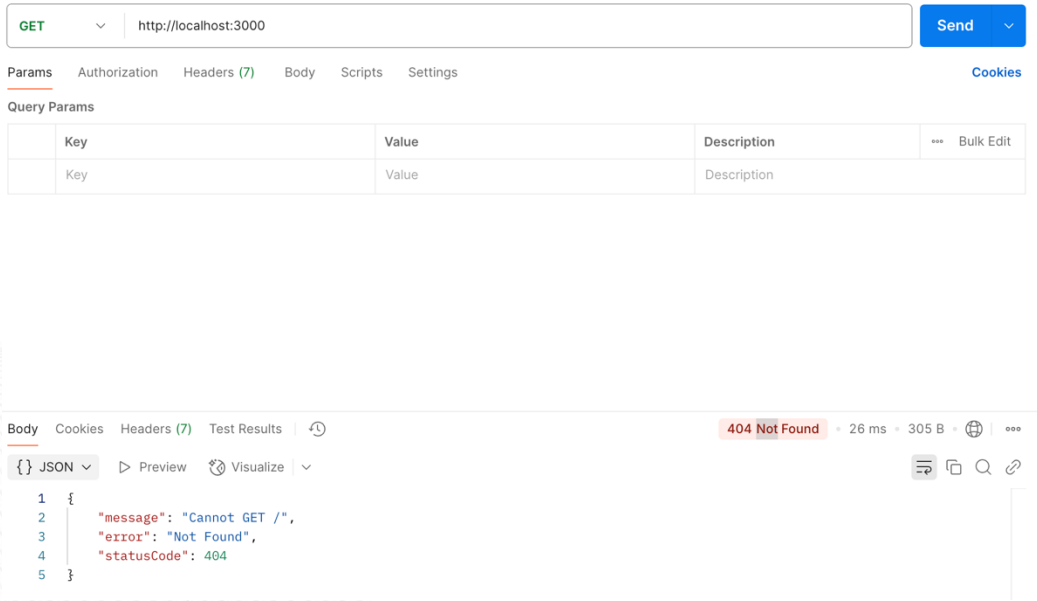
Método HTTP	Decorador en NestJS	Ejemplo de ruta
GET	@Get()	/users
POST	@Post()	/users
PUT	@Put()	/users/:id
PATCH	@Patch()	/users/:id
DELETE	@Delete()	/users/:id

Controladores

Hace un momento vimos como el decorador @Module convirtió una clase básica en un objeto que tienen cierta funcionalidad, con esto vamos a ver como aparecen servicios, módulos y otras cosas nuevas que dependiendo de su decorador, así mismo sabremos de que se esta hablando pero sin dejar de escribir código como clases.

Vamos ahora a poner en practica ya que recordemos que los controladores son los que controlan rutas, son los encargados de escuchar la solicitud y emitir una respuesta. Ej: Rutas CRUD.

Lo vimos en Postman,



Hacemos una solicitur HTTP tipo GET

Y el controlador nos devuelve un mensaje diciendo,
Que esta ruta no se encuentra.

https://docs.nestjs.com/controllers

Controladores

Vamos a hacer un controlador, recordando que instalamos un CLI global al inicio. **nest -h**

```
ingdiegoc@iMac-de-Diego project-primer-project % nest -h
Usage: nest <command> [options]

Options:
  -v, --version                                   Output the current version.
  -h, --help                                      Output usage information.

Commands:
  new|n [options] [name]                          Generate Nest application.
  build [options] [apps...]                       Build Nest application.
  start [options] [app]                           Run Nest application.
  info|i                                          Display Nest project details.
  add [options] <library>                         Adds support for an external library to your project.
  generate|g [options] <schematic> [name] [path]  Generate a Nest element.
    Schematics available on @nestjs/schematics collection:
```

name	alias	description
application	application	Generate a new application workspace
class	cl	Generate a new class
configuration	config	Generate a CLI configuration file
controller	co	Generate a controller declaration
decorator	d	Generate a custom decorator
filter	f	Generate a filter declaration
gateway	ga	Generate a gateway declaration
guard	gu	Generate a guard declaration
interceptor	itc	Generate an interceptor declaration
interface	itf	Generate an interface
library	lib	Generate a new library within a monorepo
middleware	mi	Generate a middleware declaration
module	mo	Generate a module declaration
pipe	pi	Generate a pipe declaration
provider	pr	Generate a provider declaration
resolver	r	Generate a GraphQL resolver declaration
resource	res	Generate a new CRUD resource
service	s	Generate a service declaration
sub-app	app	Generate a new application within a monorepo

Controladores

La imagen anterior nos muestra como existe un alias para cada cosa que queramos crear, así que vamos a usar co para crear el controlador, teniendo en cuenta que siempre es una buena practica, darle una estructura ordenada a nuestros proyecto, y en NestJs nos recomiendan esta.

Estructura de módulo recomendado:

```
src
- common
—— decorators
—— dtos
—— filters
—— guards
—— interceptors
—— middleware
—— pipes
— common.controller.ts
— common.module.ts
— common.service.ts
```

Controladores

Siguiendo la estructura vamos a crear el modulo, según las ayudas de nest, debemos escribir entonces:
nest g mo path/nombre del modulo

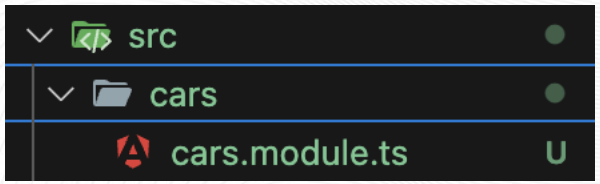
Pero, como estoy en la raíz del proyecto simplemente el path será el de src así que escribiré este comando para crear un api de carros.

```
ingdiegoc@iMac-de-Diego project-primer-project % nest g mo cars
```

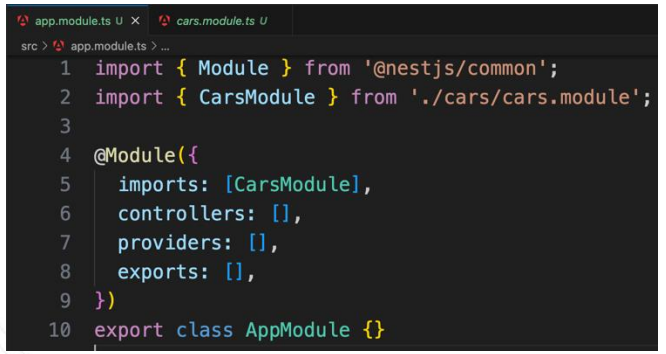
Una vez le de enter, la consola me mostrara que se creo y actualizo app.module.ts

```
ingdiegoc@iMac-de-Diego project-primer-project % nest g mo cars
CREATE src/cars/cars.module.ts (81 bytes)
UPDATE src/app.module.ts (206 bytes)
```

Y en src me aparece:





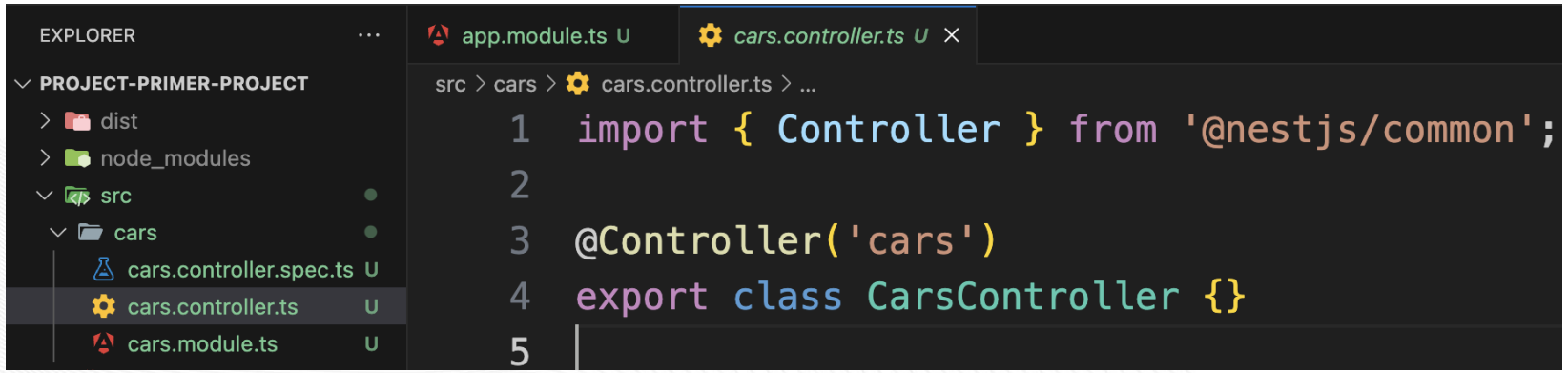


Controladores

En este momento creamos un modulo vacío, así que vamos a crear ahora si el controlador.

```
ingdiegoc@iMac-de-Diego project-primer-project % nest g co cars
CREATE src/cars/cars.controller.spec.ts (478 bytes)
CREATE src/cars/cars.controller.ts (97 bytes)
UPDATE src/cars/cars.module.ts (166 bytes)
```

Y es todo para crear el controlador, miremos src/cars



Controladores

Creemos un endpoint con un decorador @Get() para crear un endpoint de prueba así:

app.module.ts U | cars.controller.ts U

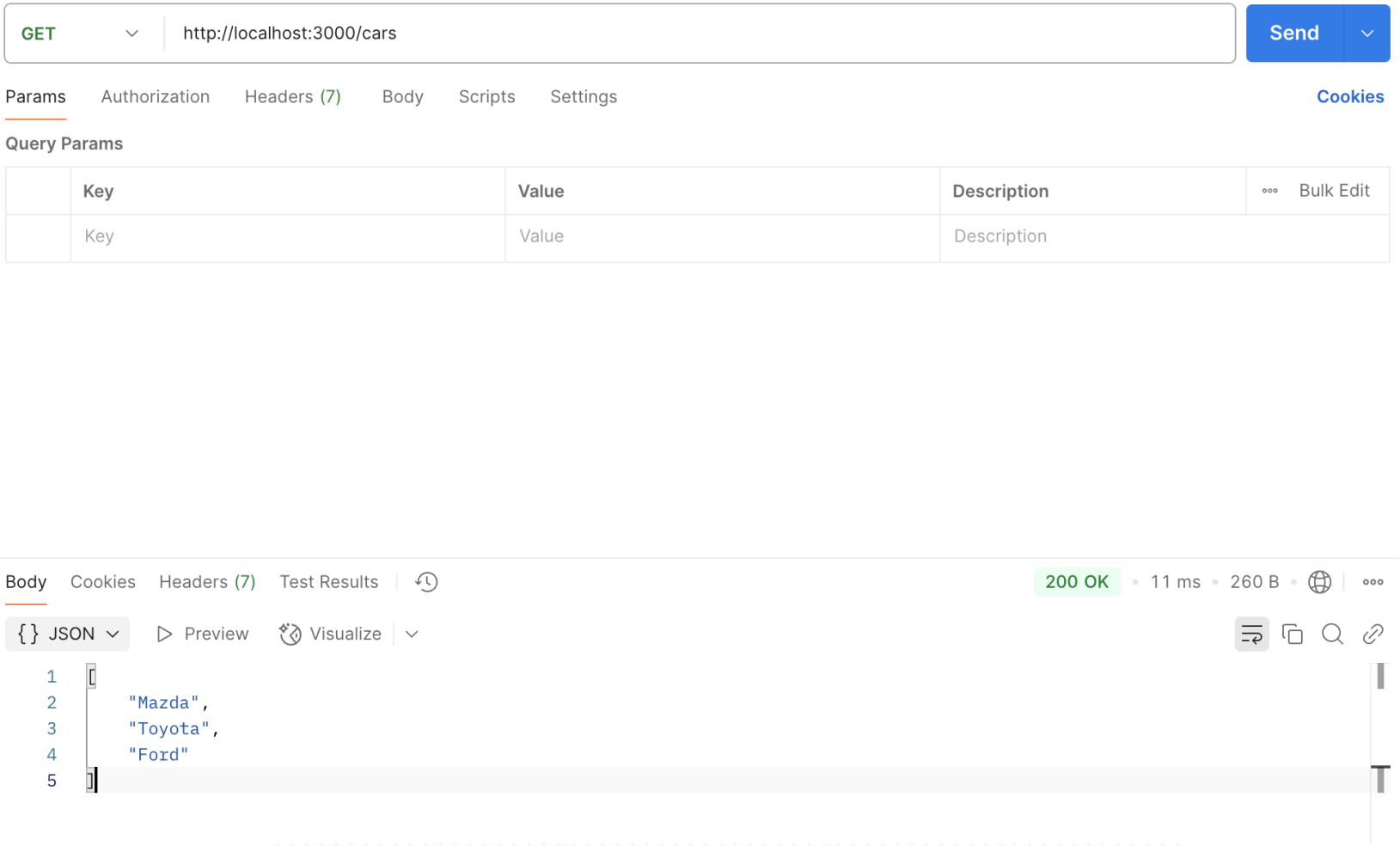
src > cars > cars.controller.ts > ...

```
1  import { Controller,Get } from '@nestjs/common';
2
3
4
5  @Controller('cars')
6  export class CarsController {
7
8      @Get()
9      getAllCars(){
10         return ['Mazda', 'Toyota', 'Ford']
11     }
12 }
```

El decorador de la línea 8 es de suma importancia ya que sin el, el controlador no sabra que tipo de solicitud HTTP esta llegando del lado del cliente.

Controladores

Ahora, para llamarlo desde postman usamos la ruta por defecto junto con el nombre del modulo que acabamos de crear



Decoradores

Los **decoradores** son funciones especiales de **TypeScript** que se usan con el símbolo @ y permiten **añadir metadatos a las clases, métodos, propiedades o parámetros**.

Tipos de Decoradores en NestJS

NestJS usa diferentes tipos de decoradores, que se pueden dividir en cuatro categorías principales:

1. **Decoradores de clases**
2. **Decoradores de métodos**
3. **Decoradores de parámetros**
4. **Decoradores de propiedades**

Decoradores

Decoradores de Clases

Se aplican a **clases enteras** y se utilizan para definir módulos, controladores, servicios y proveedores.

```
1  import { Controller } from '@nestjs/common';
2
3  @Controller('cars') // Define el prefijo de la ruta "/cars"
4  export class CarsController {}
```

Este decorador indica que CarsController es un controlador que manejará las rutas /cars

Decoradores

Ejemplo de decorador @Module():

```
1  import { Module } from '@nestjs/common';
2  import { CarsController } from './cars.controller';
3  import { CarsService } from './cars.service';
4
5  @Module({
6    controllers: [CarsController], // Define los controladores del módulo
7    providers: [CarsService], // Define los servicios del módulo
8  })
9  export class CarsModule {}
```

@Module() define un módulo en NestJS, agrupando controladores y servicios.

Decoradores

Decoradores de Métodos

Se aplican a **métodos dentro de una clase** y definen el comportamiento de las rutas HTTP en los controladores.

Decorador	Método HTTP
@Get()	GET
@Post()	POST
@Put()	PUT
@Patch()	PATCH
@Delete()	DELETE

Decoradores

Ejemplo de un controlador con rutas HTTP:

app.module.ts U | cars.controller.ts U

src > cars > cars.controller.ts > ...

```ts
import { Controller, Get, Post, Put, Delete } from '@nestjs/common';

@Controller('cars')
export class CarsController {

  @Get()
  getAllCars() {
    return ['Toyota', 'Ford', 'BMW'];
  }

  @Post()
  createCar() {
    return 'Coche creado';
  }

  @Put()
  updateCar() {
    return 'Coche actualizado';
  }

  @Delete()
  deleteCar() {
    return 'Coche eliminado';
  }
}
```

Decoradores

Decoradores de Parámetros

Se aplican a los parámetros de los métodos y se usan para capturar **valores de la URL, cuerpo, query string, headers, etc.**.

Decorador	Descripción
@Param()	Obtiene parámetros de la URL
@Query()	Obtiene valores del query string
@Body()	Obtiene datos del cuerpo de la solicitud
@Headers()	Obtiene headers de la solicitud

Decoradores

Decoradores de Parámetros

Se aplican a los parámetros de los métodos y se usan para capturar **valores de la URL, cuerpo, query string, headers, etc.**.

```typescript
import { Controller, Get, Post, Param, Query, Body } from '@nestjs/common';

@Controller('cars')
export class CarsController {

  @Get(':id') // Ruta dinámica: /cars/10
  getCarById(@Param('id') id: string) {
    return `Carro con ID: ${id}`;
  }

  @Get()
  getCarByBrand(@Query('brand') brand: string) {
    return `Buscando carros de la marca: ${brand}`;
  }

  @Post()
  createCar(@Body() body: any) {
    return `Coche creado con datos: ${JSON.stringify(body)}`;
  }
}
```

@Param(), @Query(), y @Body() permiten extraer datos de la solicitud.

Decoradores

Así se ven:

Tipo de Decorador	Ejemplo	Uso
Decorador de Clase	@Controller('cars')	Define una clase como controlador
Decorador de Método	@Get(), @Post()	Define rutas HTTP
Decorador de Parámetro	@Param(), @Body()	Captura datos de la solicitud
Decorador de Propiedad	@Injectable()	Inyección de dependencias

Decoradores

Como ya vimos en teoría como usar y que son los decoradores, pues ahora vamos a buscar un carro por ID.

Por ahora y ya que no se en que otro lugar colocar mis propiedades voy a configurar así mi controlador.

app.module.ts U | cars.controller.ts U

src > cars > cars.controller.ts > ...

```ts
import { Controller, Get } from '@nestjs/common';

@Controller('cars')
export class CarsController {


    private cars = ['Mazda', 'Toyota', 'Ford']

  @Get()
  getAllCars() {
    return this.cars;
  }
}
```

Decoradores

Como ya vimos en teoría como usar y que son los decoradores, pues ahora vamos a buscar un carro por ID.

Por ahora y ya que no se en que otro lugar colocar mis propiedades voy a configurar así mi controlador. Recordemos correr la consola con **npm run start:dev** para no tener problemas y detener la consola en cada cambio.

app.module.ts U | cars.controller.ts U

src > cars > cars.controller.ts > ...

```ts
import { Controller, Get } from '@nestjs/common';

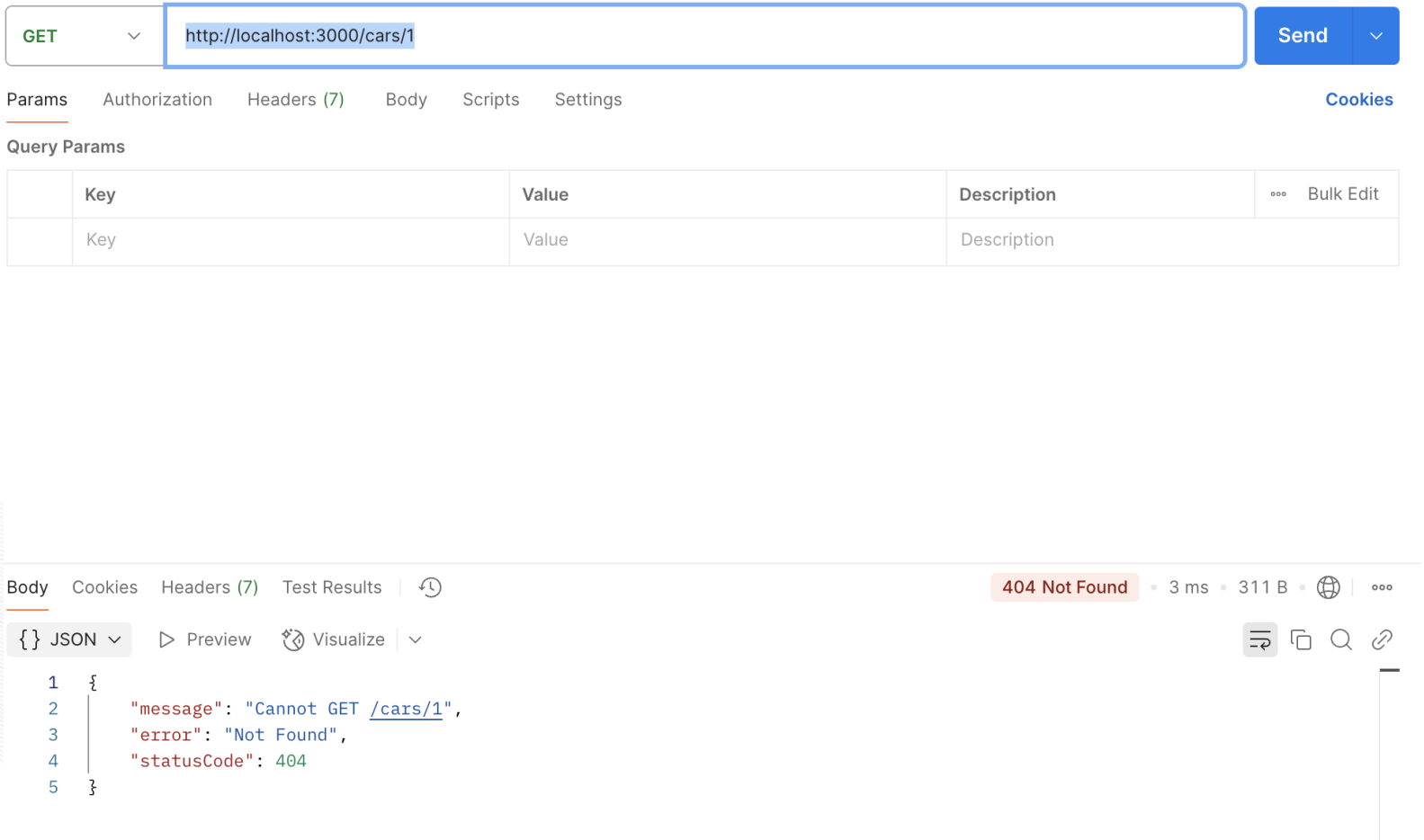
@Controller('cars')
export class CarsController {


    private cars = ['Mazda', 'Toyota', 'Ford']

  @Get()
  getAllCars() {
    return this.cars;
  }
}
```

Decoradores

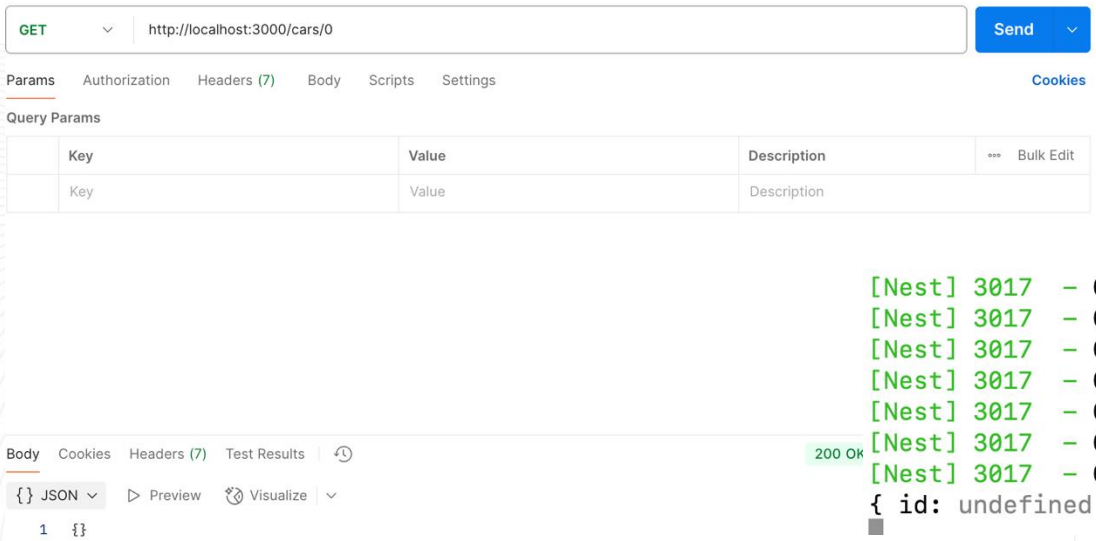
Ahora, lo que queremos hacer es buscar el carro ‘Mazda’, recordemos que en este array el espacio empieza en 0 y si voy a buscar con el método GET en postman en la ruta http://localhost:3000/cars/1 me dirá directamente que no existe ya que Nest tiene una capa de manejo de excepciones automática que va a entrar a funcionar en este tipo de errores inesperados.



Decoradores

Voy a crear un método que me ayude con lo que busco que es encontrar un carro por ID

```
  @Get(':id')
  getCarByID(id) {
    console.log({ id });
    return {
      id,
    };
  }
}
```

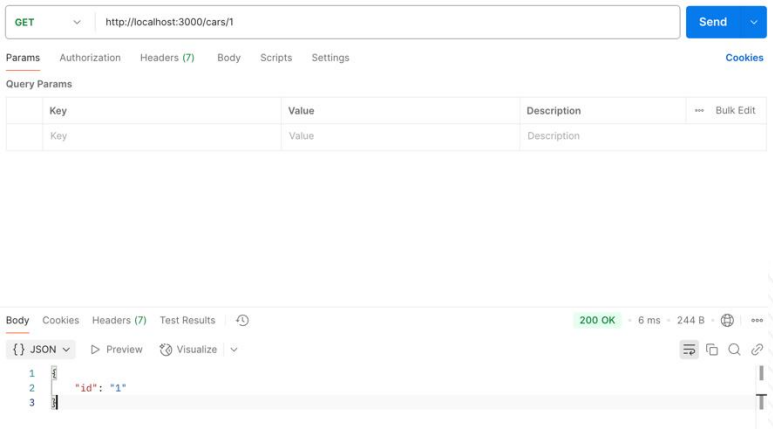



```
[Nest] 3017  - 03/16/2025, 4:56:37 PM     LOG [NestFactory] Starting Nest application...
[Nest] 3017  - 03/16/2025, 4:56:37 PM     LOG [InstanceLoader] AppModule dependencies initialized +10ms
[Nest] 3017  - 03/16/2025, 4:56:37 PM     LOG [InstanceLoader] CarsModule dependencies initialized +1ms
[Nest] 3017  - 03/16/2025, 4:56:37 PM     LOG [RoutesResolver] CarsController {/cars}: +4ms
[Nest] 3017  - 03/16/2025, 4:56:37 PM     LOG [RouterExplorer] Mapped {/cars, GET} route +2ms
[Nest] 3017  - 03/16/2025, 4:56:37 PM     LOG [RouterExplorer] Mapped {/cars/:id, GET} route +1ms
[Nest] 3017  - 03/16/2025, 4:56:37 PM     LOG [NestApplication] Nest application successfully started +2ms
{ id: undefined }
```

Decoradores

Como vimos anteriormente, aun no se encuentra nada, pero se reconoce el parámetro que se le pasa por url. Para esto, veamos los decoradores mas arriba y decoremos la propiedad ID tal como debe ser. (Recuerda importar Param en nestjs/common)

```
  @Get(':id')
  getCarByID( @Param('id') id) {
    console.log({ id });
    return {
      id,
    };
  }
}
```



Decoradores

Ejercicio

Hacer que se imprima el valor que corresponda al id de cualquier carro dentro del array y también hacer el caso en que se ingresen letras.

Servicios

¿Qué son los Servicios en NestJS?

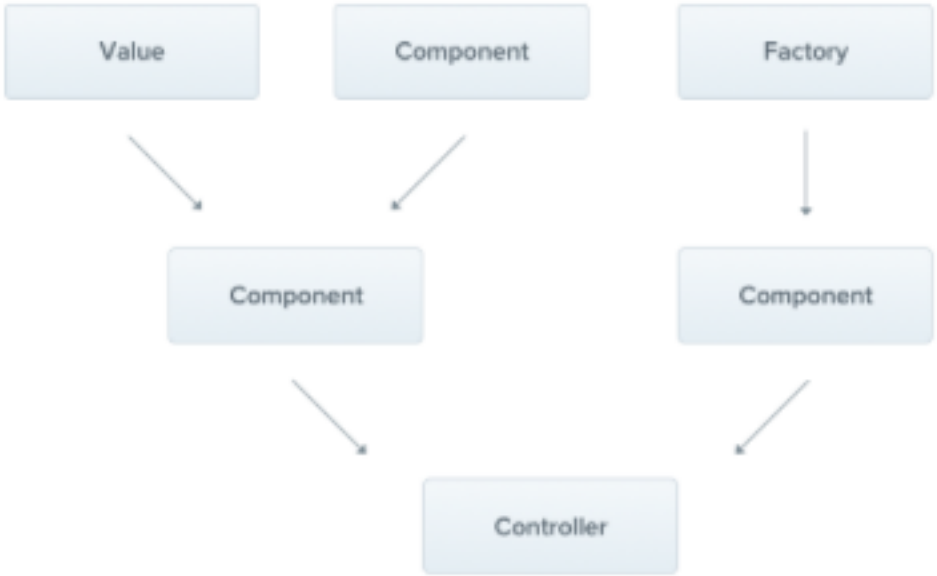
Los **servicios** en **NestJS** son clases que contienen la **lógica de negocio** y pueden ser **inyectados** en controladores u otros servicios. Su propósito es separar la lógica de las rutas (controladores) y mantener un código limpio y modular.

¿Por qué usar Servicios?

- ✔ Separan la lógica de negocio del controlador.
- ✔ Permiten la reutilización del código en diferentes partes de la aplicación.
- ✔ Facilitan la inyección de dependencias en NestJS.
- ✔ Mejoran la mantenibilidad del código.

Servicios

En este momento, todos los datos están en nuestro controlador, pero eso no es del todo correcto ya que pensemos en que pasa si se necesitan esos datos en otros controladores? No se podrían tomar, así que necesitamos tener un lugar donde podamos poner la lógica del negocio. Es por esto que usaremos un servicio ya que Alojan la lógica de negocio de tal manera que sea 🚗 reutilizable mediante inyección de dependencias. Ej: PeliculasService para todo lo relacionado a obtener, grabar, actualizar o eliminar información de películas. (https://docs.nestjs.com/providers#services)

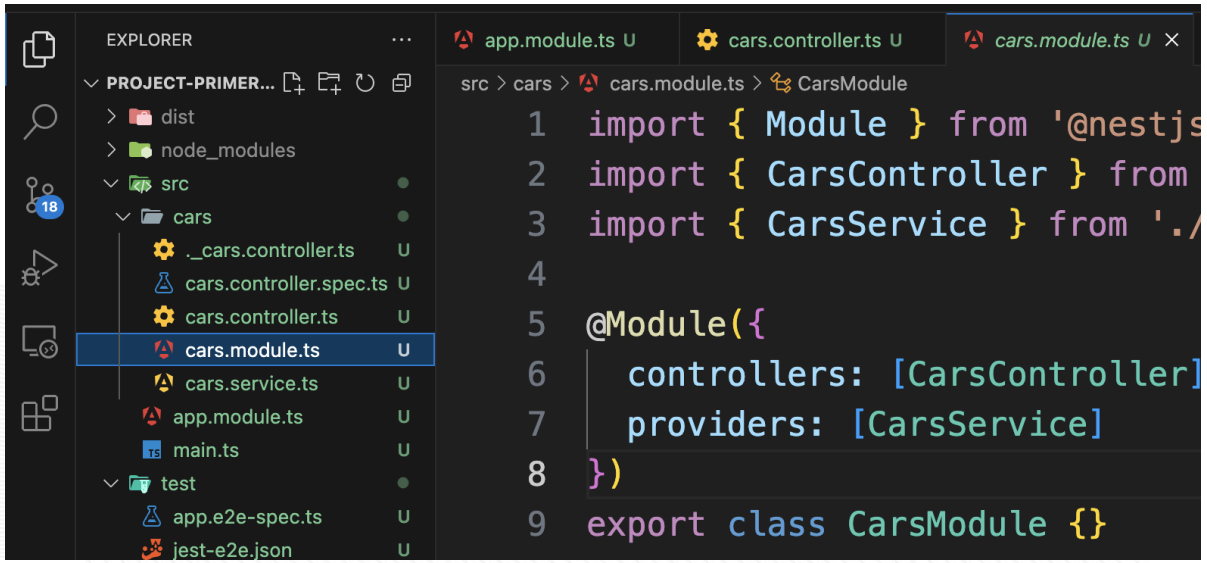


Servicios

Tal como vemos en la documentación no todos los providers son servicios, es decir que no todos los provider tiene una lógica de negocio. En pocas palabras, vamos a usar los providers cuando necesitemos inyectar una dependencia y un servicio cuando necesitemos insertar una inyección de la lógica de negocio. Con **nest –h** buscamos la ayuda y creamos un nuevo servicio. El - -no-spec es para que no cree el archivo de pruebas.

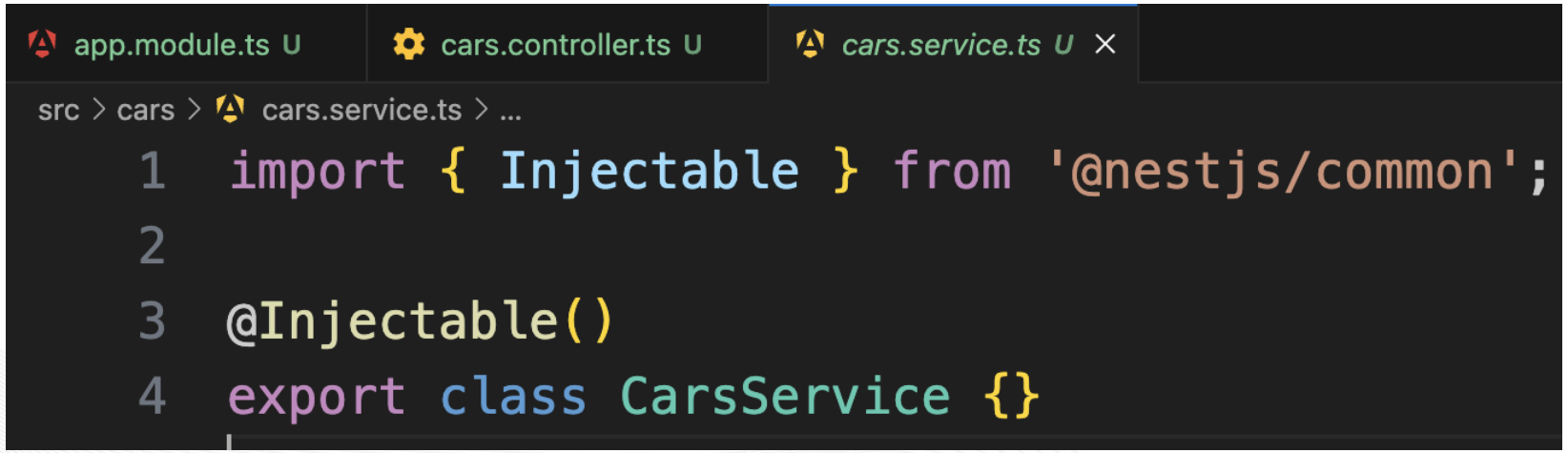
```
ingdiegoc@iMac-de-Diego project-primer-project % nest g s cars --no-spec
CREATE src/cars/cars.service.ts (88 bytes)
UPDATE src/cars/cars.module.ts (240 bytes)
```

Cuidado. Acaba de crear un cars.services.ts y acaba de modificar car.modules.ts



Servicios

Y en el archivo cars.service.ts encontramos una clase común y corriente con el decorador @Injectable() que básicamente lo que dice es que se permite inyectar datos ahí.



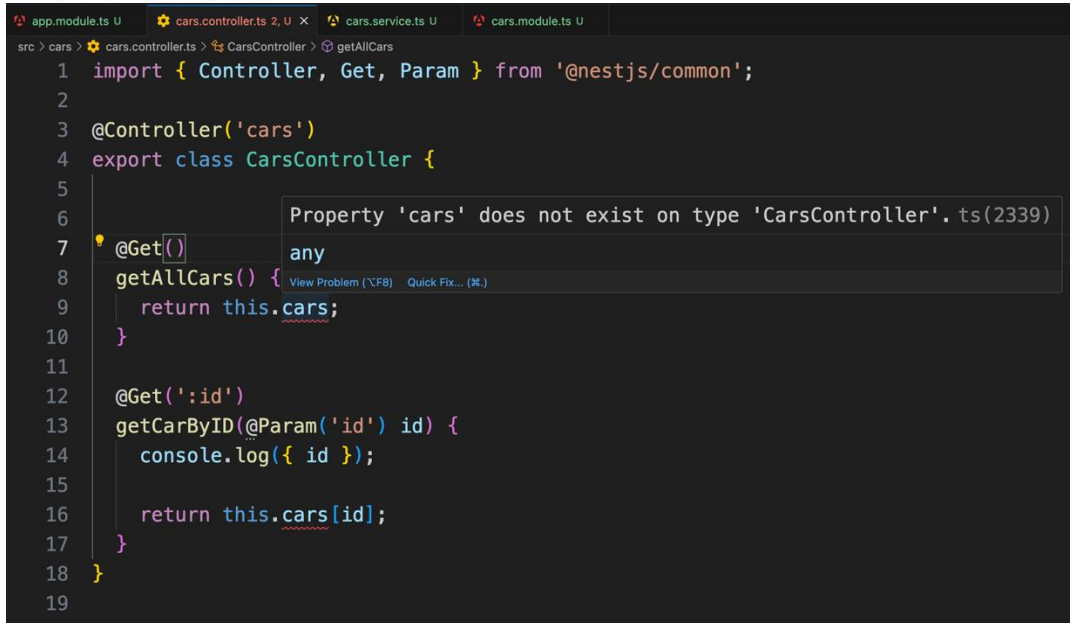
Servicios

Recordemos entonces que para que funcione debe estar dentro de provider en el archivo de módulos. Ahora, ¿Cómo poder hacer para tener la lógica dentro de nuestro provider?
Lo primero es que me llevo la propiedad privada del controlador y la pego en el servicio, pero la modifico ahora así.



Inyección de dependencias

Resulta que el controlador nos muestra ahora un error



Y justamente nos dice que la propiedad no existe por lo tanto mucho menos su referencia.

Inyección de dependencias

Es por esto que vamos a inyectar la dependencia de cars.services en cars.controller y esta inyección se hace en el constructor así.

app.module.ts U | cars.controller.ts 2, U | cars.service.ts U | cars.module.ts U

src > cars > cars.controller.ts > CarsController > getCarByID

```ts
import { Controller, Get, Param } from '@nestjs/common'
import { CarsService } from './cars.service';

@Controller('cars')
export class CarsController {

  constructor(
    private readonly carsService: CarsService
  ){}

  @Get()
  getAllCars() {
    return this.cars;
  }

  @Get(':id')
  getCarByID(@Param('id') id) {
    console.log({ id });

    return this.cars[id];
  }
}
```

Inyección de dependencias

Pero ahora tenemos el problema que si intentamos acceder a la propiedad, no tenemos datos.

```
@Get()
getAllCars() {
  return this.carsService;
}
```

Y esto es normal por que en el servicio, tengo las propiedades privadas, así que creare unos métodos públicos que me permitan la lectura de los datos. En el servicio creo

```
findAll(){
  return this.cars;
}
```

Y lo llamo en el controlador asi:

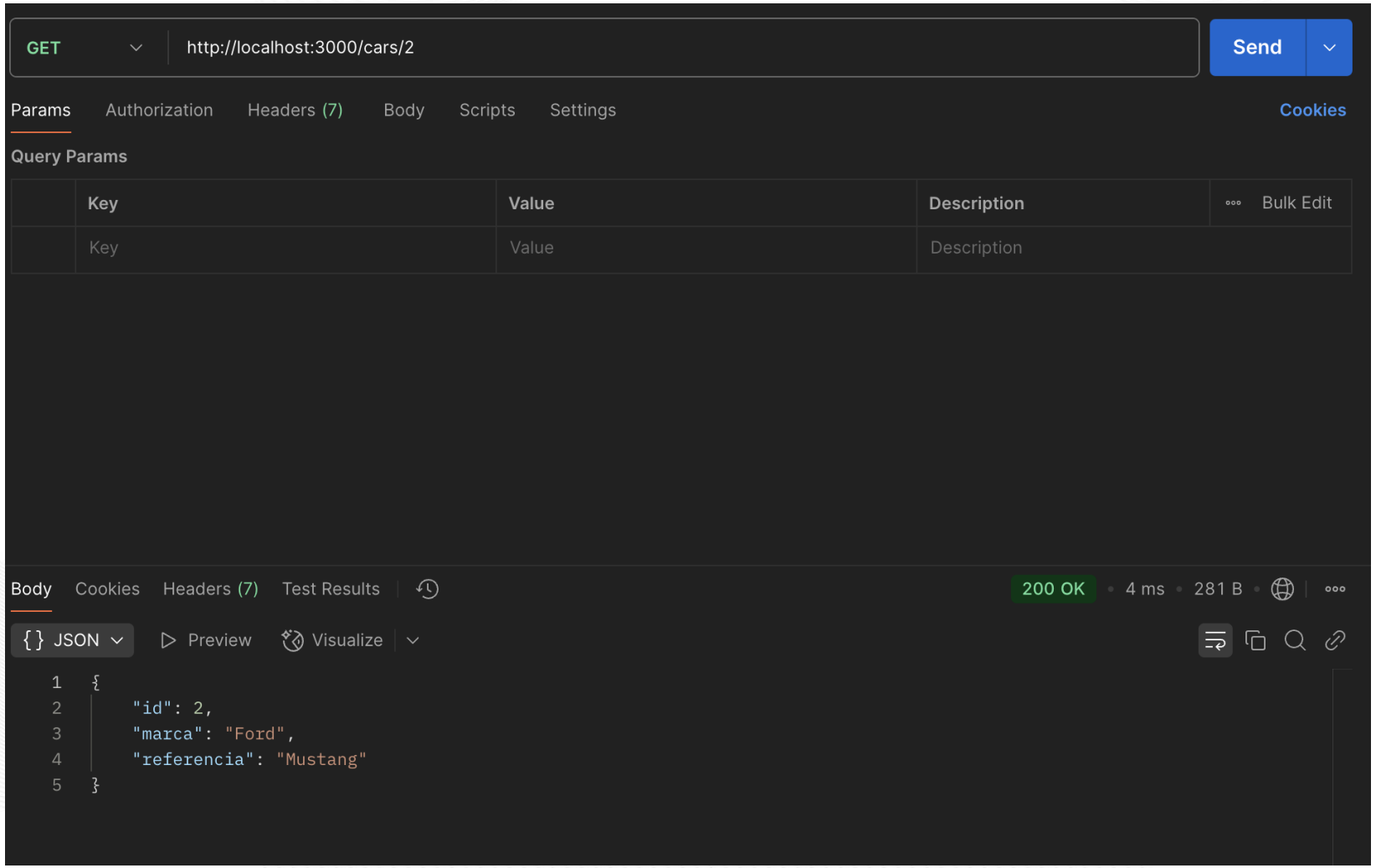
```
@Get()
getAllCars() {
  return this.carsService.findAll();
}
```

```
@Get(':id')
getCarByID(@Param('id') id) {
  console.log({ id });

  return this.carsService.findAll();
}
```

Inyección de dependencias

Ahora creemos un un método que se llame findOneById(id:number) y van a retornar el carro que coincida con ese id



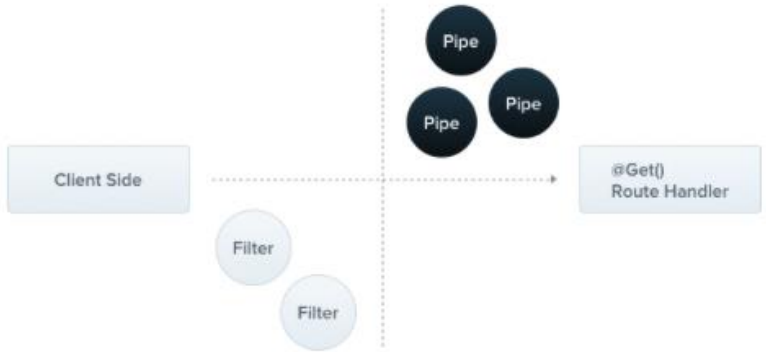
Pipes

¿Qué son los Pipes en NestJS?

En **NestJS**, los **pipes** son una **funcionalidad clave** utilizada para transformar y validar datos antes de que sean procesados por el controlador.

Un pipe puede: **Validar** datos (ejemplo: asegurarse de que un campo es un número).

✅**Transformar** datos (ejemplo: convertir un string a número).
✅ **Rechazar** solicitudes con datos incorrectos.
https://docs.nestjs.com/pipes



Pipes

Existen algunos integrados por defecto.

Pipes integrados por defecto

`ValidationPipe`	`ParseIntPipe`
`ParseBoolPipe`	`ParseArrayPipe`
`ParseFloatPipe`	`ParseUUIDPipe`

Tenga en cuenta que todos los que dicen parse, se refieren a que van a transformar algo, y el único que dice validation, es por que esta destinado a la validación.

Pipes

Apliquemos un ejemplo dentro del cars.controller.

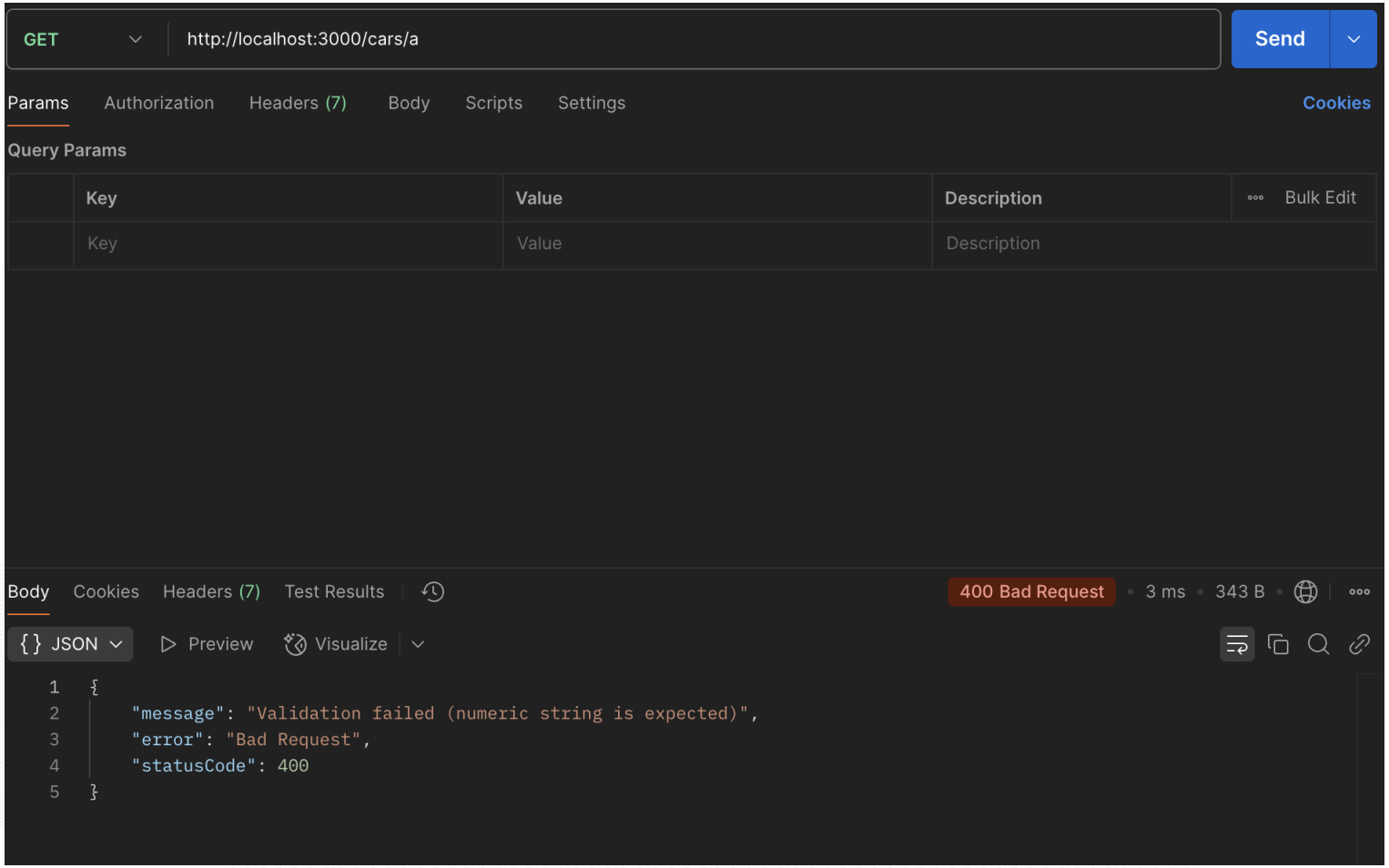
```
@Get(':id')
getCarByID(@Param('id') id:string) {
  console.log({ id });
  return this.carsService.findOneById(+id);
}
```

Actualmente tengo este método así, pero veamos como podemos hacer para transformar ese id, ya que así en este momento yo cambie el tipo de string a number, el seguirá siendo un string.

```
@Get(':id')
getCarByID(@Param('id', ParseIntPipe) id:number) {
  console.log({ id });
  return this.carsService.findOneById(id);
}
```

Pipes

Pero y que pasa si en la url no le envio un numero si no un carácter?
https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status



Exceptions Filter.

¿Qué son los Exception Filters en NestJS?

Los **Exception Filters** en **NestJS** son una forma de manejar **errores globales** y personalizar la respuesta cuando ocurre una **excepción** en la aplicación.

Por defecto, NestJS maneja los errores con una respuesta genérica en **JSON**.
Si no controlas los errores, obtendrás respuestas como esta:

```
{
    "statusCode": 500,
    "message": "Internal server error"
}
```

Exceptions Filter.

La forma mas rápida de aplicarlo es así:

```
findOneById(id: number) {
  const car = this.cars.find(car => car.id === id);

  if (!car){
      throw new NotFoundException(`El carro con el id: ${id} no esta disponible`)
  }
}
```

```
{
    "message": "El carro con el id: 4 no esta disponible",
    "error": "Not Found",
    "statusCode": 404
}
```

Exceptions Filter.

Realice un crud completo usando todo lo visto anteriormente. Por ahora solo quiero que muestre en el body de postman un texto que muestre que su método se ejecuto y sobre que id se ejecuto.

GRACIAS

Línea de atención al ciudadano: 01 8000 910270
Línea de atención al empresario: 01 8000 910682

@SENAComunica

www.sena.edu.co